

What machine learning taught us about the open stacks problem, and what it did not

Alexandre Linhares*

28 September 2026

Abstract

The minimization of open stacks problem (MOSP) asks in what order to make a set of products so that the number of customer orders open at any one time is as small as possible. It is NP-hard, and it is equal to a classical graph quantity: the pathwidth of a graph on the customers, plus one. Over four unattended analysis loops we pointed machine learning and large-scale measurement at a corpus of 6,376 instances with certified optimal values and at 51,000 generated instances. Learning never made the solver faster: three learned components (bounds, branching, imitation) each resolved into a statement about the solver or the problem instead. What the measurements established is more useful. The optimum is sandwiched between two textbook graph numbers, both now proved theorems; the best point estimate is a treewidth heuristic; the solver’s proved lower bound fails for a structural reason that no formula over the usual invariants can repair. Hardness has a sharp ridge at a simple density condition, grows exponentially along it, and depends on the labelled graph and on nothing else in the matrix. The imitation ranker distilled into a one-sentence closing rule that beats it. A differential harness found a false refutation the verification machinery could not see, and every refutation at 40 customers or fewer now carries a certificate a third party checks in milliseconds. The paper states what size range each claim covers, which of the pre-registered kill criteria were met, and what remains unchecked at 125×125 .

1 The problem, and why it is a graph problem

The problem. A factory has a list of products to make and a list of customers. Each customer wants some subset of the products. A customer’s *stack* opens when the first product they want is made and closes when the last one is done. Products are made one at a time, in some order. The question is which order keeps the largest number of simultaneously open stacks as small as possible. That number, minimised over all orders, is the instance’s *optimum*. The problem is NP-hard [13]. The input is a 0/1 matrix with one row per customer and one column per product.

The graph. Draw one vertex per customer and join two customers whenever they share a product. Every product then becomes a *clique* (a set of vertices all joined to each other), and the graph is a union of cliques. This is the *MOSP graph* of Yanasse [18], and the optimum depends only on it: two matrices with the same MOSP graph have the same optimum. The literature also uses a second graph, with a vertex per product and an edge between products that share a customer; Yanasse and Senne [19] call the two “completely different” and use the second one for one purpose only, splitting an instance into independent parts.

Pathwidth. A *path decomposition* of a graph is a sequence of overlapping vertex sets (“bags”) laid out along a line, such that every edge has both ends in some bag and the bags containing any one vertex form a contiguous stretch. Its width is the size of the largest bag minus one, and the *pathwidth* of the graph is the least width of any path decomposition. An equivalent picture is *vertex separation*: order the vertices left to

*The analyses reported here were carried out with Claude Code in unattended loops; see “How this work was done” in Section 1. Every number in the paper is copied from a numbered section of the project’s findings file and regenerates by a named command (Appendix A).

right, and at each cut count the vertices on the left that still have a neighbour on the right; the vertex separation number is the smallest achievable maximum count. Kinnersley [10] proved the two are equal. *Treewidth* is the same construction with the bags arranged along a tree instead of a line; it is never larger than pathwidth.

The chain. The equality between open stacks and pathwidth rests on four published steps. MOSP is the same problem as gate matrix layout, a VLSI problem, by definition [13], whose interval-graph reading goes back to Ohtsuki et al. [15], Wing et al. [17] and Möhring [14]; gate matrix layout cost equals pathwidth plus one [7], through a column-expansion lemma [6] that is Yanasse’s clique-per-product graph in matrix vocabulary; pathwidth equals vertex separation [10], whose NP-completeness is due to Lengauer [12]; and the graph in question is the MOSP graph [18]. Related search-game formulations are in Kirousis and Papadimitriou [11]. So:

$$\text{optimum} = \text{pathwidth}(\text{MOSP graph}) + 1 = \text{vertex separation}(\text{MOSP graph}) + 1.$$

We had the two graphs swapped. For part of its life the project attached this theorem to the product graph. Measured against published optima it undercounted, which was read as a gap in the reduction; it was a consequence of testing the wrong graph. The mistake went further than the code. The Lean 4 development’s original statement of the reduction was over the product graph too, and over that graph the statement is false: one product shared by four customers, each with a private product of their own, has optimum 4, while its product graph is a star with pathwidth 1. That counterexample is now a Lean theorem, the false statements are deleted, and the equality above is a `sorry`-free theorem in both directions over the MOSP graph [§ 26]. Two more statements in the development turned out to be false rather than hard: a “branch lemma” that asked for neither connectivity nor attachment, refuted by four isolated vertices [§ 39]. The corrected lemma and its separator form are proved [§ 39, 40]. At the end of the work the development holds 34 named theorems and one `sorry`, kept on purpose as a stated conjecture [§ 40]. The lesson we record is that a stated-but-unproved theorem needs a small-instance check before it is committed.

How this work was done. The analyses were carried out with Claude Code in four unattended loops, one item per fresh session, each item an immutable question with a pre-registered kill criterion, a progress file and a report section as its deliverable. The first plan asked ten questions (findings [§ 1–14]), the second eleven ([§ 15–27]), the third seven ([§ 28–40]). Every number in this paper is copied from the section it cites and regenerates by a command named there; Appendix A lists them. The paper computes nothing new. Section references of the form [§ *n*] point to the findings file; citations in the usual form point to the literature.

2 The corpus and the instruments

What “certified” means. A *witness* is a production order that achieves a value *k*; it is checked by simulating the order and counting stacks, which anyone can do. A *refutation* is a proof that no order achieves *k* − 1. An optimum is *certified* when it has both, or when a proved lower bound already equals *k*. The corpus holds 6,376 instances at 9 to 134 customers, every one with a certified optimum and a witness, drawn from the standard benchmark collections (Harvey, Simonis, Faggioli–Bentivoglio, Chu & Stuckey, Shaw, Wilson, SCOOP, Challenge) [§ 1].¹ The lopsidedness matters: 5,938 of the 6,376 have 30 or fewer customers, and the class that costs a day of compute is 25 rows [§ 28].

The corpus is smaller than it looks. Two instances with the same MOSP graph up to renaming have the same optimum. Counted by graph, the 6,376 instances hold 3,667 distinct graphs; 42.5% of the corpus is isomorphic copies, and 1,669 instances (26.2%) are complete graphs whose optimum is trivially the number of customers [§ 1]. 157 graph classes span more than one benchmark file, so a train/test split by file leaks copies across the split; every split in this work groups by file and by isomorphism class together [§ 1]. That leak was later measured: it was worth about a seventh of a learned heuristic’s recorded gain [§ 28].

¹Two 125 × 125 entries had their optimality claims withdrawn after the bug of Section 5 and are being re-certified; their values remain verified upper bounds [§ 35].



Figure 1: The instance space of the corpus, a UMAP embedding of 6,376 instances on 28 structure features, coloured by collection (left), by number of customers (middle) and by collection again on size-free features (right). The map is a lattice of streaks, one per (n, m) size cell, not a continuum; within a streak the generators separate. Chu & Stuckey and the industrial SCOOP instances share the sparse corner [§ 2].

The generators are fingerprintable. A random forest on 28 structure-only features assigns 94.4% of instances to the collection that generated them with the source file held out, and 98.8% among instances of equal size where chance is 61.6% [§ 2]. The fingerprint can be stated in two lines: every Harvey instance has constant row or column sums and no other generator’s does; Faggioli–Bentivoglio caps the largest product at half the customers; Chu & Stuckey is sparse where nothing else is [§ 2]. Figure 1 shows the instance space: a lattice of 74 size cells with empty regions between, and the 24 industrial SCOOP instances in the same sparse corner as all 200 Chu & Stuckey instances [§ 2]. A result on one collection is a result about that generator.

The generated campaign. Small instances are free: at 40 customers a random instance is generated, solved and refuted in a fraction of a second. So the questions about density were asked on ensembles we control. The campaign holds 37,800 instances in 252 cells (10 to 40 customers, products equal to or twice the customers, two generators, 150 per cell), every one certified and refuted under both search configurations, for 2.16 core-hours [§ 9, 10]. It was then extended upward: 6,747 certified instances at 50–75 customers [§ 16], 2,750 at other product ratios [§ 25], 3,378 more at 40–100 [§ 37], and one 25-instance cell at 100 customers on the hardness ridge, of which one instance certified [§ 34]. Every instance regenerates byte for byte from its seed.

Nodes, not seconds. The complete search of Chu and Stuckey [4], which the solver runs, visits search *nodes*: partial closing orders. Node counts are the unit of hardness throughout. At 40 customers or fewer the refutation is 0.5–0.7% of the wall-clock cost, seconds are a smooth function of size alone, and node counts vary by three orders of magnitude across cells at fixed size, so at these sizes the clock does not see hardness at all [§ 10]. On a loaded machine the clock also invents speed-ups: in one study seconds spread $1.49\times$ where nodes spread $1.05\times$ [§ 18]. Every node count is dated against the version of the search that produced it, because a pruning rule changed mid-way (Section 5).

The rules. Four rules held for all four loops. A prediction is never a bound: nothing learned reaches the code that decides which k to try, because a lower bound one too high makes the solver return a wrong answer whose witness still verifies. Every split groups by file and isomorphism class. Every claim states its size range, and nothing below 100 customers is evidence about 125×125 except through an extrapolation that says so. No solver default was changed by a loop; the one code change that altered an answer, the bug fix of Section 5, went through the owner.

3 What determines the optimum

The best point estimate is a treewidth heuristic. Thirteen pathwidth-adjacent graph invariants were added to the feature table. One carries almost everything: the min-fill treewidth heuristic plus one. Table 1 compares it with the solver’s own proved bounds. It is exact on 85.7% of the corpus, better than either bound, and leads

Table 1: Point estimates of the optimum over the whole corpus (9–134 customers, 6,376 instances). MAE is mean absolute error in stacks. The treewidth heuristic is not a bound [§ 4].

estimate	exact	MAE
min-fill treewidth +1	85.7%	0.188
solver’s upper bound (heuristic)	84.6%	0.239
solver’s lower bound (proved)	77.0%	0.431

Table 2: The 338 corpus instances where the proved lower bound is two or more below the optimum, by whether pathwidth exceeds treewidth. Exact treewidth was computed to 26 vertices by subset dynamic programming and to 64 by a budgeted decision search; “open” means the search returned an interval [§ 21].

	instances	share
pathwidth > treewidth (bound structurally blocked)	164	48.5%
pathwidth = treewidth	80	23.7%
open (82 of them above 64 customers)	94	27.8%

in every size band up to about 50 customers; at 125 it is off by 4.2 stacks on average and inside its ± 1 band on 2 of 25 instances [§ 4, 14]. It is a bound on nothing: it sits below the optimum on 481 instances and above on 428 [§ 4]. On the residual over the proved bound the whole group of invariants adds nothing (grouped MAE -0.004): they know what the bound knows, expressed differently [§ 4].

A sandwich, now two theorems. Two textbook graph numbers bracket the optimum on every instance we have. The *degeneracy* of a graph is the largest d such that some subgraph has every vertex of degree at least d . A *bandwidth* estimate (reverse Cuthill–McKee) is the longest edge when the vertices are laid on a line by that heuristic. On all 6,376 instances,

$$\text{degeneracy} + 1 \leq \text{optimum} \leq \text{bandwidth}_{\text{rcm}} + 1,$$

with the two ends coinciding, and so forcing the optimum, on 2,823 instances [§ 5, 26]. Both inequalities are proved in the Lean development for every finite graph, through vertex separation and Kinnersley’s equality, and since the equality of Section 1 is itself proved they are theorems about the optimum rather than about pathwidth [§ 26]. Treewidth plus one is a lower bound by the same route, proved through a lemma that Mathlib lacked, that the path graph is a tree [§ 39]. Where the two ends differ, the optimum’s position between them is not a constant: pooled over the corpus it sits at the midpoint, but by density it runs from the bottom below 1.5 customers per product to two thirds of the way up above 6 [§ 5, 12].

Where the proved bound fails, and why. The solver’s lower bound is the best of four components: a trivial bound, a clique bound, contraction degeneracy [2], which is the arc-contraction bound of Yanasse et al. [20] under another name, and a neighbourhood-expansion bound, which is the classical vertex-isoperimetric bound of Harper [9] as applied to pathwidth by Chandran and Kavitha [3]. It equals the optimum on 77.0% of the corpus, misses by one on 17.7%, and by two or more on 338 instances (5.3%; 51.0% of Chu & Stuckey) [§ 6]. A gap of two never occurs below 20 customers [§ 6]. The reason is structural, not statistical. Three of the bound’s components are really bounds on *treewidth*, and on about half of the 338 gap instances pathwidth is provably larger than treewidth (Table 2), so no tuning of those components could ever close the gap [§ 6, 21]. The instances look like small cliques glued at a few hub customers into a branching tree, with a fringe of customers who want a single product [§ 6].

The smallest such graph has ten vertices. A bit-flip search with the exact solver as oracle found a 10-vertex, 21-edge graph, three K_4 s glued pairwise at three hub vertices plus a tenth vertex joined to one non-hub of each, with treewidth 3 and pathwidth 5, and proved it vertex- and edge-minimal for a gap of two [§ 21]. A census then settled the question by exhaustion: over all 287,884 graphs on at most 9 vertices the gap is 0 or 1; exactly four graphs on 10 vertices have gap 2, all one family; 1,034 on 11 have it and none has gap 3, so gap 3

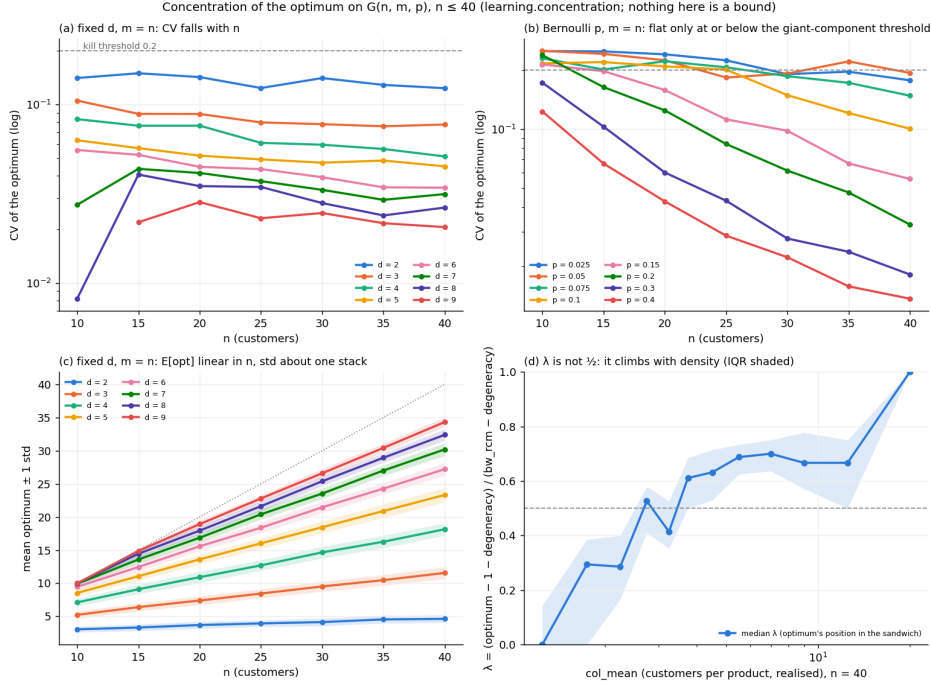


Figure 2: Concentration of the optimum on generated random instances at 10–40 customers. (a) At fixed customers per product the coefficient of variation falls with size; (b) under Bernoulli sampling it stays flat only at or below the giant-component threshold; (c) the mean optimum is a line in n with a standard deviation of about one stack; (d) the optimum’s position in the sandwich climbs with density and is not one half. Nothing here is a bound [§ 12].

needs at least 12 vertices [§ 27]. Twelve of the 1,034 have treewidth 2 and pathwidth 4 with no K_4 at all, three triangles strung between two poles: a second shape, not a tree of cliques, that any pathwidth bound must see [§ 27].

No formula and no branching rule fixes it. Exact treewidth plus one beats the solver’s certified bound on 933 corpus instances (14.6%) and on 199 of the 338 gap instances [§ 24]. Beyond that, nothing was found. A search over 11,589 two-invariant formulas on 30 branching-aware invariants left 47 survivors, 36 of which were treewidth plus one restated; 77% of fitted candidates leaked on a grouped hold-out, because a constant fitted on 51,000 instances is the maximum of a sample; no candidate beat $\max(\text{bound}, \text{treewidth} + 1)$ on more than 0.1% of the instances with exact treewidth [§ 24]. A harness that takes any candidate bound as a function, checks it on 49,862 certified instances, attacks it with 11,880 adversarial evaluations and scores it on the gap instances was then built [§ 38]. Three branching-aware candidates, resting on the branch lemma of Fellows and Langston [6] and Kinnersley [10] and its separator generalisation, went through it: all valid, all survived the adversary, and by branching none beat the reference on any gap instance; the kill criterion (none beats the reference on more than 5% of gap instances) was met at 0.0/0.0/2.4% [§ 38]. The next bound idea has to be a different pathwidth argument.

On random instances the optimum concentrates. At every one of the 36 cells at 40 customers the coefficient of variation of the optimum is under 0.2 (median 0.031); a cell is two or three adjacent integers [§ 12]. The mean is a straight line in the number of customers at fixed degree, with $r^2 \geq 0.999$ in all 17 fixed-degree series [§ 12] (Figure 2). A closed-form fit for that mean has held-out error 0.49 stacks on the 252 cell means, and it is an estimate of a mean, never a bound: per instance it sits below the optimum on 9,445 and above on 10,641 [§ 12]. Its constants break with size. On Chu & Stuckey’s instances from 30 to 125 the line holds but the intercept and slope do not, so the bias grows to +8.1 stacks on the sparsest 125-customer class while staying within ± 2 at densities 4–10 [§ 14]. Every confidence interval calibrated at 40 customers or fewer fails by 50–75 and is not to be quoted at 50 or more; that kill criterion was met for every interval [§ 14].

MOSP hardness map: nodes to refute optimum – 1 over (n , customers per product); orange squares mark each row's peak; 150 instances per cell

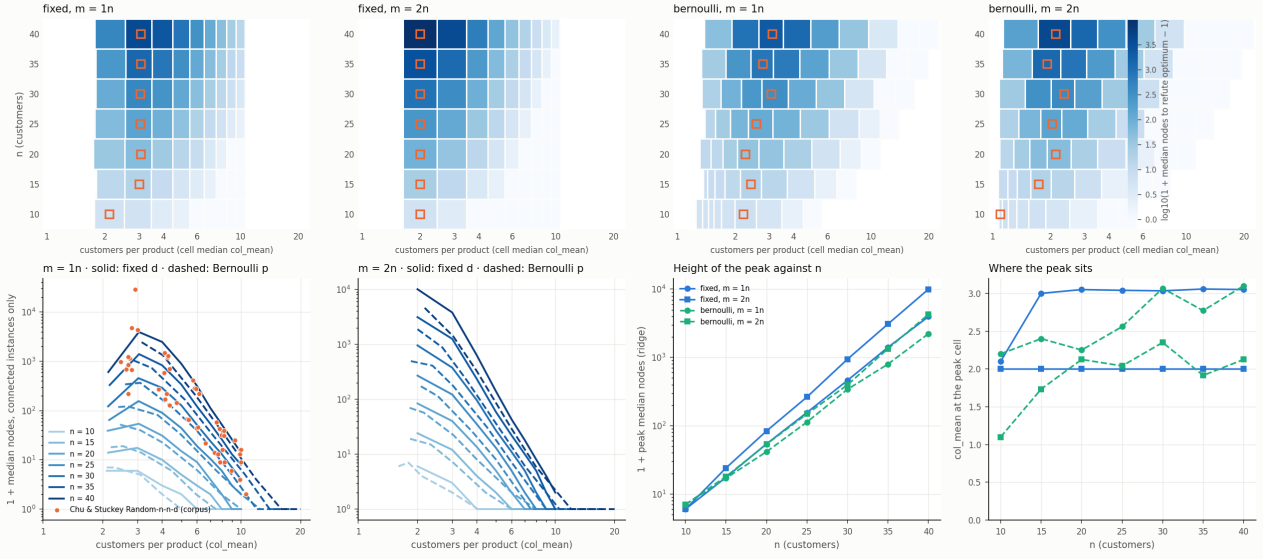


Figure 3: The hardness map. Top: median $\log_{10}$ nodes to refute optimum – 1 over (customers, customers per product) for two generators and two product ratios, 150 instances per cell, with each row's peak marked. Bottom left: the same at $m = n$ as curves, with Chu & Stuckey's corpus instances at 30 and 40 customers overlaid; they sit on the peak. Bottom right: the peak's height grows exponentially in n and its location is fixed [§ 11].

4 What makes an instance hard

There is a ridge. Figure 3 is the one picture the rest of this section explains. At every size from 15 to 40, in both generators, the median number of nodes to refute optimum – 1 rises and falls with density by two orders of magnitude each side of a peak at 40 customers, and the peak sharpens with size [§ 11]. On the ridge the median grows exponentially at $0.086\text{--}0.105 \log_{10}$ nodes per customer, doubling every 2.9–3.5 customers, with a power law ruled out at 5–10 times the residual; off the ridge growth is exponential too but slower, doubling every 4.5–4.9 customers [§ 11]. The ridge sits at about three customers per product when products equal customers and two when products are twice the customers [§ 11]. Chu & Stuckey's "density 2" classes, the ones that take a day at 125×125 , are the peak cell at both 30 and 40 customers, and their density 4 is its dense shoulder [§ 11].

The growth rate fell, and then did not. Fitted at 40 customers or fewer, the ridge law pre-registered 5×10^{11} nodes for a 125×125 ridge refutation; the two counts then on record were $1.6\text{--}1.7 \times 10^{11}$, within a factor of three after eight decades of extrapolation [§ 11, 14]. The campaign to 75 customers found the rate drifting down in 18 of 18 fixed-density series, by about 0.002 per customer per ten customers, and a drift-corrected law put the figure at 1.5×10^{11} [§ 16]. The 100-customer ridge cell contradicted the drift: its refutations sit $0.5\text{--}0.7$ decades above the drifting law's median, and 24 of 25 did not finish in 2,400 s each after $3.4\text{--}4.8 \times 10^9$ nodes [§ 34]. Fitted as a censored cell, the rate over 75 to 100 is $0.116 [0.110, 0.121]$ per customer against $0.092\text{--}0.095$ over 60 to 75 [§ 35]. Read like with like (the drift-corrected headline had compared a law for one search configuration against counts from another), a constant-rate law refitted on 10 to 100 gives Table 3's figure: a median of 2.7×10^{11} nodes and 55 hours on one core per 125×125 ridge refutation, with the record's three counts falling 0.2 decades either side [§ 35]. Every 125 figure is an extrapolation of 25 customers from a cell that is mostly lower bounds. Figure 4 shows the fits against the corpus counts.

Where the ridge is. The obvious coordinate, mean degree of the MOSP graph, is not it: the mean degree at the peak runs from 3.9 to 37.6 as the product ratio goes from twice the customers to an eighth of them [§ 25]. The coordinate that is the same across every ratio is the *cover excess*, the number of ones in the matrix minus the number of products, per customer: $2.0\text{--}2.5$ at every fixed- degree peak and $1.8\text{--}3.0$ at every Bernoulli peak, the only candidate within a factor of 1.65 across ratios [§ 25] (Figure 5). At excess 1 the customer–product

Table 3: Successive predictions of the median node count for a 125×125 ridge refutation (Chu & Stuckey density 2), each from the sizes it was fitted on. The last row is the one to quote; it is for the search configuration before the bug fix of Section 5, and post-fix counts are at least $2.34 \times$ larger on the 100 cell [§ 35].

fitted on	law	prediction at 125	section
10–40	constant rate	5×10^{11}	[§ 11]
10–75	drifting rate	1.5×10^{11}	[§ 16]
10–100	constant rate, like with like	2.7×10^{11} [2.3, 3.2]; 55 h	[§ 35]
record	three counts, $\log_{10}$ nodes	11.21, 11.22, 11.66 (law’s median 11.43 [11.36, 11.51])	[§ 35]

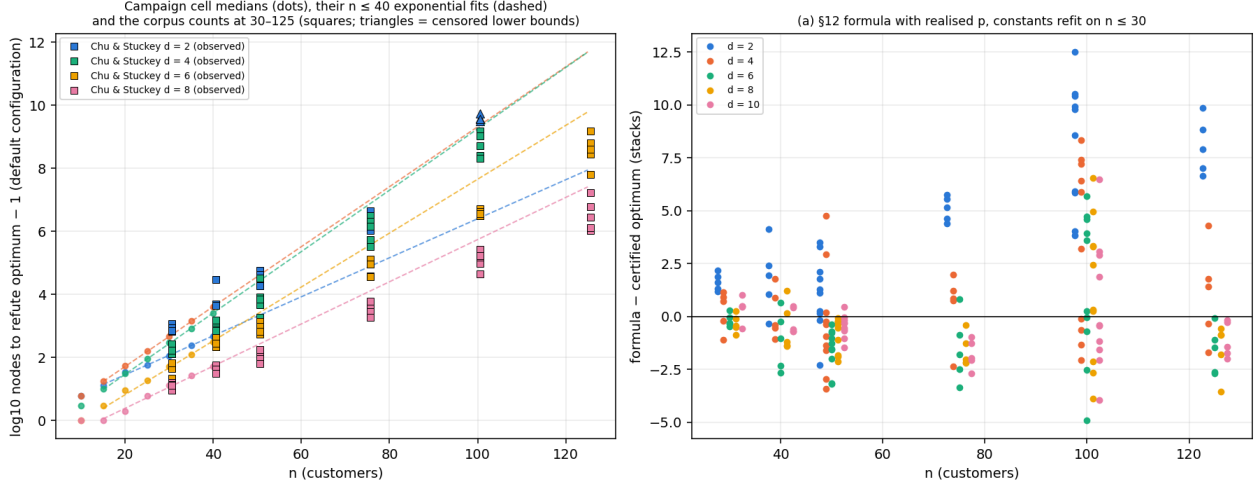


Figure 4: Do the small-size laws survive to 125? Left: campaign cell medians at 10–40 customers with their exponential fits (dashed) against the corpus counts at 30–125 (squares; triangles are censored lower bounds), by Chu & Stuckey density class. The forms hold; the fitted rates overstate the dense classes and land within a factor of three on the two day-long ones. Right: the mean-optimum formula’s error grows linearly with size on the sparsest class and stays within about ± 2 elsewhere [§ 14].

incidence graph has as many edges as vertices; the ridge is one independent cycle per customer. In customers-per-product terms the ridge is at about $1 + 2.4n/m$ [§ 25]. It is a density condition, not a percolation threshold: the giant component of the incidence graph appears 0.34–0.78 decades below it, and no core or connectivity threshold coincides with it at every ratio [§ 36]. What it is a transition of is the search’s own state space: the number of closed customer sets *reachable* from the empty set through steps that fit the budget equals the node count to 0.03–0.06 decades at the median and peaks where the nodes peak [§ 36].

How high the ridge is. The excess locates the ridge but says nothing about its height, which needs the number of products separately. With 57 series peaks under each configuration the law that survives is $\log_{10} \text{nodes} \approx -0.10 + n(0.089 + 0.058 \log_{10}(m_{\text{eff}}/n))$, where m_{eff} counts products that carry an edge after dominance; its in-sample error is 0.18 decades and 0.21 leaving a series out, and the excess at the peak adds nothing to it [§ 37].

Only the labelled graph matters, for the search. Re-covering the same edge set with different products, 15,900 re-coverings of 1,400 instances, leaves the default node count *exactly* unchanged in every pair: the search reads neighbour masks and nothing else [§ 13]. Graph- only features predict nodes as well as graph plus matrix. What does move the count is renaming the customers: 2–6% at the median, up to $2.26 \times$ under one configuration and $8.1 \times$ under the other, the search’s tie-breaks and the floor for any label-free predictor [§ 13]. For the SAT path the story is different: re-covering moves the solver’s conflict count by 0.26 – $1.67 \times$ between the 10th and 90th percentiles, in the direction of the formula’s clause count, with which conflicts correlate at Spearman 0.93–0.98 across instances [§ 30]. Hardness is a property of the labelled graph for the search and of the formula size for SAT.

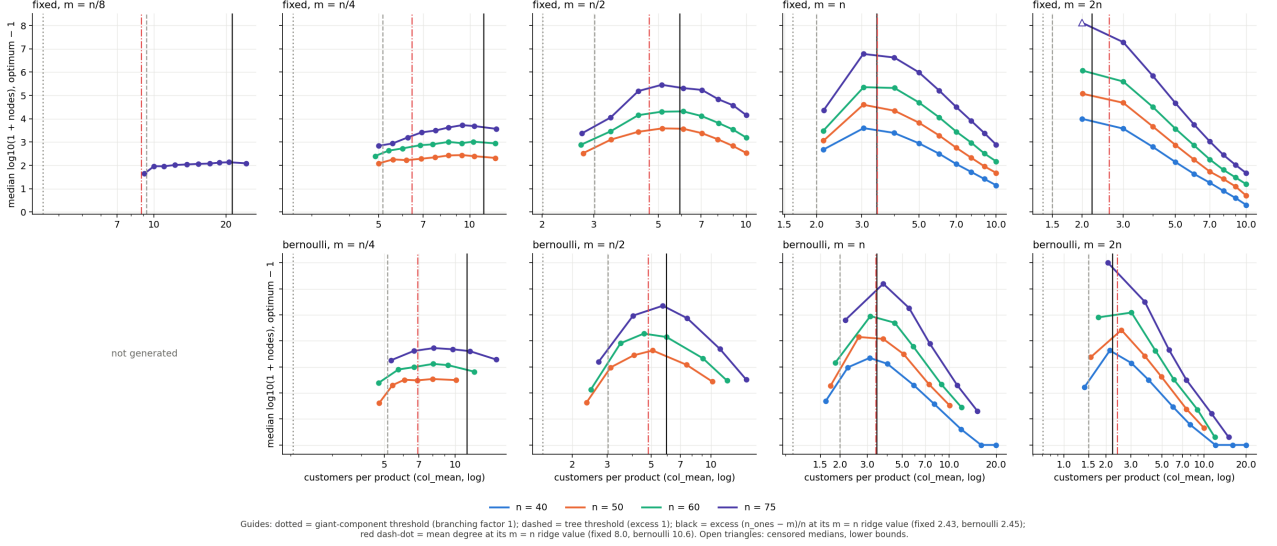


Figure 5: Why the ridge is where it is. Median $\log_{10}$ nodes against customers per product at 40–75 customers, one panel per product ratio from $m = n/8$ to $m = 2n$. Vertical guides: the giant-component threshold (dotted), excess 1 (dashed), mean degree calibrated at $m = n$ (red), and cover excess calibrated at $m = n$ (black). Only the excess line lands on the peak in every panel [§ 25].

The cost can be predicted within a decade. A censored regression in log space on label-free graph features, trained at 10–75 customers, puts 87.8% of the 98 default-configuration counts and 89.3% of the 103 other-configuration counts at 100–125 within one decade of the truth, against 57.5–65.4% and 59.2–61.2% for the two earlier laws, with a bias of +0.4 decades [§ 19]. It sized the 100-customer ridge cell correctly from the heuristic upper bound and under-predicted by 1.1 decades from the certified optimum, because the optimum’s fraction of n is its strongest term [§ 34]. It is a prediction for ordering a queue, never a bound.

5 What the solver learned about itself

The two-key rule. The project had trained a LightGBM ranker to imitate the closing orders of the witnesses, and it helped. Distilling it produced a rule a person can state: *close the customer that opens the fewest new stacks; on ties, the one with the most unclosed neighbours* [§ 7]. Table 4 compares it with what it distilled from. The first key is the search’s own cheapest-first fan order, so the greedy is the search’s first leaf; the tie-break is the reverse of the textbook rule’s [§ 7]. Run over the whole corpus as the seed of the restricted depth-first search, the rule beats the learned seed on every statistic, and does so with no model, no training and no dependency, which closed the question of whether LightGBM belonged in the solver [§ 28]. It is not the literature’s heuristic under another name: the Minimal Cost Node heuristic of Becceneri et al. [1], reimplemented from its pseudocode and reproducing its worked example and 21 of 21 published values in Frinhani et al. [8], shares the rule’s first key on 98.8% of its decisions and has the opposite tie-break; the two produce the same closing order on 0.6% of instances and the rule wins 592 to 242 where they differ [§ 29].

The rule against Chu and Stuckey’s own start. It helps to say plainly what Chu and Stuckey [4]’s solver does, because the rule changes only one part of it. The solver works in two phases. First it looks for a good production sequence quickly: it builds one greedily, one customer at a time, and then spends a fixed budget (200,000 steps of a restricted depth-first search) trying to improve it. That gives an upper bound, a number of stacks known to be achievable. Then it proves the answer: it asks “can it be done with one stack fewer?” and keeps asking until the answer is no. The last “no” is the proof of optimality, and on hard instances it is almost all of the cost.

The two-key rule replaces only the greedy start of the first phase. Chu and Stuckey start from the textbook minimal-cost-node order; we start from the rule’s order and leave the restricted search and the whole proof phase untouched. So the rule cannot change any certified answer, only how close the first phase gets and how

Table 4: Closing-order constructions, held out (1,920 instances, 9–134 customers, top) and as the seed of the restricted DFS over the whole corpus (6,376, bottom). MAE and overshoot are in stacks above the optimum [§ 7, 28].

construction	MAE	exact	worst
two-key rule	0.348	80.7%	+9
LightGBM ranker	0.519	75.0%	+34
MCN (textbook, node-closing)	1.616	50.2%	+26
DFS seed	mean overshoot	exact	ms per instance
rule + cs-dfs	0.092	94.2%	16.7
learned + cs-dfs	0.128	92.1%	20.6
cs-dfs alone	0.241	84.5%	29.5

Table 5: The two-key rule against Chu and Stuckey’s own start. Top: the first phase (finding a good sequence) over all 6,376 certified instances. Bottom: the whole solve, finding plus proving, on four large instances, each run on its own processor core; C&S is Chu and Stuckey’s start. “Nodes” are the positions the search visited; M is millions [§ 28].

<i>First phase, 6,376 instances</i>				
start	at the optimum	mean excess	worst excess	ms per instance
Chu & Stuckey (MCN)	84.5%	0.241	+10	29.5
two-key rule	94.2%	0.092	+8	16.7
rule better on 780 instances, worse on 12, equal on 5,584				
<i>Whole solve, four large instances</i>				
instance	optimum	start C&S / rule	nodes C&S / rule	seconds C&S / rule
Random-100-100-4-1	46	49 / 47	248.2 M / 247.8 M	135 / 137
Random-125-125-6-1	81	84 / 83	68.3 M / 68.3 M	52 / 58
Random-125-125-6-3	80	85 / 83	286.9 M / 286.9 M	200 / 199
Random-125-125-6-4	80	83 / 80	400.0 M / 400.0 M	269 / 269

much work the second phase is left with.

Over all 6,376 certified instances the rule’s start ends the first phase at the optimum on 94.2% of them, against 84.5% for Chu and Stuckey’s, and it is better on 780 instances and worse on 12 (Figure 6). Where it loses, it loses by one stack on 11 of the 12; where it wins, it often closes a gap of two or three stacks entirely, and in one case turns ten stacks too many into two. It is also faster, 16.7 against 29.5 milliseconds per instance, because a better starting sequence lets the restricted search discard more of what it would otherwise explore [§ 28].

Figure 7 shows the same comparison by instance size. The rule leads in every band, and the lead grows with size: between 81 and 100 customers the share of instances where the first phase already finds the optimum rises from 8% to 26%. The rule does not make big instances easy. Above 100 customers its start is still about three stacks above the optimum on average, and only 27 instances of that size are in the corpus.

The whole solve tells a different story from the first phase. On 300 instances of up to 40 customers, the full two-phase solve certified the same optimum under both starts on all 300, in 2.5 seconds in total against 7.6 [§ 28]. On large instances the gain disappears. We ran four instances of 100 and 125 customers where the two starts differ by one to three stacks, each on its own processor core: all eight runs proved the right optimum, the search visited the same number of positions to within 0.2%, and the times agreed to within a few seconds, between one and four and a half minutes [§ 28]. Even where the rule starts exactly at the optimum, and so skips every “yes, that can be done” question, the saving is about eight thousand positions against four hundred million spent on the final “no”. So on large instances the rule gives a better answer if a run is stopped early, not a faster proof. Table 5 puts the two measurements side by side.

In plain English, the top half of the table says the rule is a better first guess. More than nine times in ten it already finds the best possible sequence, where Chu and Stuckey’s start does so about eight and a half times

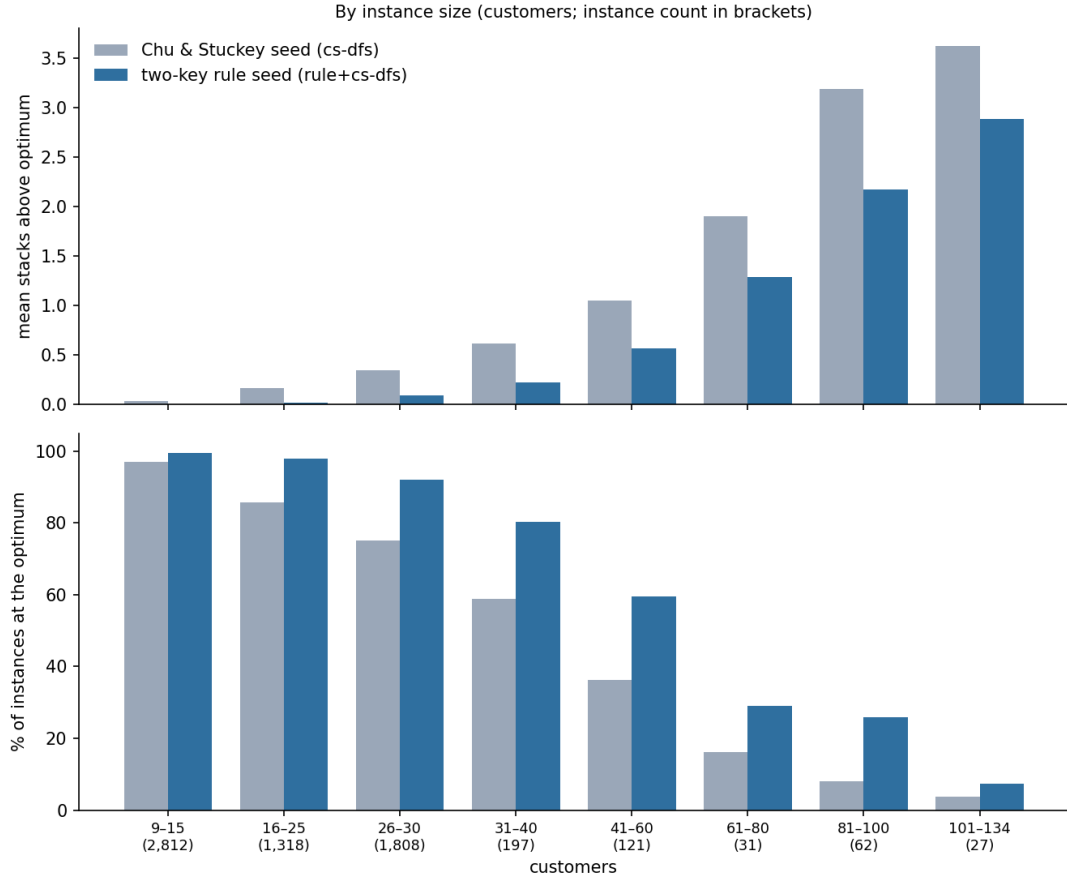


Figure 7: The same comparison by size. Top: mean stacks above the optimum at the end of the first phase. Bottom: share of instances where the first phase already finds the optimum. Grey is Chu and Stuckey’s start, blue the two-key rule’s; instance counts are in brackets under each band. Only the start differs; the proof phase is the same search under both [§ 28].

right place [§ 22].

A false refutation, found on the satisfiable side. A *dominance rule* discards a candidate move because another is provably at least as good. Such a rule may change the cost of a search and never its answer. The differential harness (Section 6) asked every certified instance at 40 customers or fewer the same question under ten labellings and two configurations, at optimum – 1 and at the optimum. Every one of 878,580 refutations held. At $k = \text{optimum}$, 88 runs on 56 instances said “impossible” [§ 15]. The stored optima were right; the C implementation of one rule, `better_move`, was wrong in a way no earlier audit could see, because a refutation one step too strong leaves the witness intact. Three bugs were involved: a wrong close count inside the rule, a cycle in which two candidates dominated each other and were discarded together, and an early exit that hid both [§ 15]. The owner fixed it. Reverting either half of the fix brings false answers back (1 and 36 of the 56 instances), so both are necessary, and no cheaper sound composition exists among the reverts [§ 31]. The fix costs $1.073\times$ the nodes at 40 customers or fewer, with the reordering carrying 81% of that; $1.54\times$ at 41–100; and at least $19.6\times$ on one 100-customer ridge instance, because at that size the over-count was doing real unsound pruning [§ 31]. One alternative sound composition is $0.854\times$ the nodes at 41–100 and a wash in seconds, proposed for tuning and not for adoption [§ 31]. Every earlier node count from that configuration is now labelled pre-fix.

6 Soundness: proofs and certificates

The blind spot. Witness verification, the corpus audit and the lower-bound guards all confirm that a value is *achievable*. None can see a refutation that is one step too strong. Three instruments now can, each with a reach.

DRAT proofs. The direct SAT encoding of the decision “at most k stacks” can log its refutation as a DRAT proof and an independent checker, drat-trim [16], verifies it from the formula alone. That was done for the corpus at 40 customers or fewer: 5,646 of 6,135 instances (92.0%), by size band 99.4/97.1/80.0/77.7% at ≤ 10 , 11–20, 21–30, 31–40 customers, for 11.2 core-hours and 11.7 GB of compressed proofs, with zero checker rejections [§ 17]. Affordability is set by the formula, roughly m^2k , not by n : the boundary $m^2k \approx 10^4$ is crossed by a 125-product instance at $k = 1$, so the direct encoding has no proof to offer at 125×125 [§ 17]. The proof costs 6,000–70,000 times what the search costs on the same question: an archive object, not a decision procedure [§ 17].

A certificate for the search. The search itself now has a proof object. A refutation is a tree of nodes; the certificate is that tree with every pruning decision named and its premise recorded. A checker sharing no code with the search recomputes every premise from the instance, checks that every candidate within budget is a child or is covered by a chain of dominations with no cycle, and replays the tree [§ 32]. On the corpus at 40 customers or fewer all 6,135 certificates verify under three configurations, agreeing with all 5,646 DRAT verdicts and covering the 489 instances the DRAT budget censored; the whole set is 5.0 MB gzipped and checks in 4.5 s, against 11.7 GB and 5,190 s for the DRAT proofs, on a different trust base [§ 32]. At 41–75 customers, with a Python emitter about $120\times$ slower than the C, 141 of 151 corpus instances verify under both configurations [§ 32]. Both bugs that shipped are rejected by the checker. The kill criterion, that a Theorem 2 step be checkable from the instance and the state alone, was not met at the step; the precise finding is that the checkable unit is the node, because whether a step prunes soundly is a property of the node’s whole step set, which is exactly what the cycle violated [§ 32]. At 125×125 the tree is 10^{11} nodes and about 10 TB, not a shippable artifact.

The differential harness to 100. Above 40 customers the harness was run under a budget: 1,294 certified instances at 50–100 customers, five labellings, both configurations, both k , 300 s per call, every call priced from a recorded run before scheduling [§ 33]. Of 25,800 calls, 12,860 of 12,860 refutations returned “impossible” and 12,938 of 12,940 witness searches simulated to the optimum, with zero disagreements [§ 33]. Refutations were priced out for eight instances, all five Random-100-100-2 and three of five Random-100-100-4: the first size at which the harness stops being affordable is 100 customers with two to four customers per product, exactly the classes that take a day at 125×125 [§ 33].

What the corpus’s optimality claims rest on. Table 6 states it plainly. At 40 customers or fewer every refutation is checked twenty ways and carries a millisecond certificate, and 92% carry a DRAT proof. At 50–100 every corpus instance but eight agrees under five labellings and two configurations. Above 100 customers nothing is checked by anything outside the search except the agreement of the two configurations [§ 22] and of a few relabellings [§ 18, 33]. Every independent audit that came free passed: the lattice oracle, which shares no code with the search, equals the certified optimum on all 2,812 instances at 15 customers or fewer; isomorphic instances carry equal optima in every class; 15,900 re-coverings solve to their base’s optimum; 5,657 SAT decisions on two backends agree with the certified value [§ 1, 8, 13, 30]. Nothing was re-certified because nothing disagreed, except the 56 instances of Section 5, whose values were right.

Table 6: How far each soundness instrument reaches, by number of customers.

instrument	reach	section
lattice oracle (no shared code)	15 (20 vectorised)	[§ 8, 23]
DRAT proof, independent checker	40, where $m^2k \lesssim 10^4$	[§ 17]
search certificate, independent checker	40 (all); 41–75 (141 of 151)	[§ 32]
differential harness, ten labellings	40	[§ 15]
differential harness, five labellings	100, except eight ridge instances	[§ 33]
two-configuration agreement only	101–125	[§ 22]

7 What learning did and did not do

The recurring pattern. The original hope was that learning from the solved instances would make the solver faster. It did not, three separate ways: a learned upper bound, a learned lower bound and learned branching each gave a measured negative, because the expensive part of proving an answer optimal is showing that one better is impossible, and better guesses do not help with that. What every learned component did instead was resolve into a finding about the solver or the problem. The imitation ranker became a one-sentence rule that beats it [§ 7, 28]. The hardness regression became a ridge with a coordinate and a height law [§ 11, 25, 37]. The bound-gap classifier became a structural explanation, the treewidth ceiling, and a proof that no formula over the usual invariants can lift it [§ 6, 24, 38]. The differential harness, built as a performance experiment, became the only tool that could see an unsound refutation, and it found one [§ 15]. The one component that did earn a place is the cost model, and its place is ordering a queue [§ 19].

The kill criteria. Each question was pre-registered with a criterion that would close it. Table 7 lists them and their verdicts. Ten were met, ten were not (one of those because the budget turned out to be below the price), and one was met on its letter and not in its spirit.

Closed, not to be rebuilt. Learned upper bounds, learned lower bounds and learned branching; imitation of any kind, single-witness or set-valued [§ 7, 8, 23]; the LightGBM dependency [§ 28]; the two-key rule as MCNh under another name [§ 29]; fan order and relabelling portfolios for refutations [§ 18, 20, 33]; a learned switch for Theorem 2 [§ 22]; a cheaper sound fix for `better_move` among the reverts [§ 31]; formulas over degree statistics or over the thirty branching invariants as new lower bounds [§ 5, 24]; branching at any separator the clique structure exposes [§ 38]; mean degree as the ridge’s coordinate [§ 25]; the ridge as a percolation threshold [§ 36]; the height as a function of the ridge coordinate [§ 37]; a sub-exponential ridge law [§ 35]; DRAT proofs for 125×125 through the direct encoding [§ 17]; and any interval calibrated at 40 customers quoted at 50 or more [§ 14].

What would count as progress next. A pathwidth lower bound that is a different argument, one that sees the separators of trees of cliques and the biconnected triangle shape of the census, on which every degree, clique and treewidth bound is provably blocked on about half the gap instances [§ 6, 21, 27, 38]; the harness is where the next candidate goes. A C emitter for the search certificate, so that the eight priced-out instances at 100 and the 125×125 classes can carry one [§ 32, 33]. The Lean gaps that remain: the correctness of the SAT encoding, the three dominance theorems the certificate checker trusts, and the minor monotonicity one bound candidate rests on [§ 32, 40]. And the price of a clean corpus at 125×125 : re-certifying the two withdrawn instances at a median of 55 hours each pre-fix, and re-refuting all eight on the fixed code at 2.34 to about $20 \times$ that [§ 35].

Table 7: Pre-registered kill criteria and their verdicts. “Met” closes the question in the negative; “not met” means the thing looked for exists.

criterion	verdict	section
No invariant moves grouped MAE by more than 0.02	met against the stated baseline (-0.012); cleared $10\times$ against structure alone	[§ 4]
Gap classifier cannot beat density and size	not met (AUC 0.931 vs 0.849)	[§ 6]
No readable rule keeps half the ranker’s gain	not met; exceeded at 115%	[§ 7]
Node counts monotone in density, no peak	not met	[§ 11]
Coefficient of variation above 0.2 up to $n = 40$	not met	[§ 12]
Graph-only features predict nodes as well as the full set	met	[§ 13]
Interval coverage collapses across the size boundary	met, for every interval	[§ 14]
Extremal search cannot beat the corpus’s worst at equal size	met for the soundness objective only	[§ 21]
100-customer ridge cells cost over 8 core-hours	met (priced at 40)	[§ 16]
drat-trim cannot be built or proofs not checkable	not met	[§ 17]
Median min-of-8 relabelling speed-up at $n \geq 60$ under $1.5\times$	met (1.003)	[§ 18]
Fewer than 80% of 100–125 counts within a decade	not met (88–89%)	[§ 19]
Fan order: paired median node change within $\pm 5\%$	met (0.0%)	[§ 20]
Learned Theorem 2 boundary agrees with the hand rule on $> 95\%$	not met (41.2%); the hand rule stands anyway	[§ 22]
Set-valued policy does not beat the rule by 0.02 MAE	met (0.008)	[§ 23]
No mined bound beats the reference on $> 5\%$ of gap instances	met	[§ 24]
MCNh’s worked example cannot be reproduced from its pseudocode	not met	[§ 29]
SAT cost as cover-independent as the search’s ($\pm 5\%$)	met on the letter (1.00), not in spirit	[§ 30]
A Theorem 2 step is not checkable locally	not met at the step; the node is the unit	[§ 32]
Certify the 100-customer ridge cell	not met; the budget was below the price	[§ 34]
No branching candidate beats the reference on $> 5\%$ of gap instances	met (0.0 / 0.0 / 2.4%)	[§ 38]

Acknowledgements

The analyses were carried out with Claude Code in four unattended loops between 25 and 28 September 2026. The exact solver follows Chu and Stuckey [4]; the benchmark instances are those of Chu and Stuckey [4], Faggioli and Bentivoglio [5] and the collections compiled in Frinhani et al. [8]. Yanasse (1997a), Becceneri, Yanasse & Soma (2004) and Yanasse (1997b) were sent by their author; Kinnersley (1992) and the two Fellows–Langston papers close the pathwidth chain on papers we hold.

References

- [1] José Carlos Becceneri, Horacio Hideki Yanasse, and Nei Yoshihiro Soma. A method for solving the minimization of the maximum number of open stacks problem within a cutting process. *Computers & Operations Research*, 31(14):2315–2332, 2004. doi: 10.1016/S0305-0548(03)00189-X.
- [2] Hans L. Bodlaender, Arie M. C. A. Koster, and Thomas Wolle. Contraction and treewidth lower bounds. *Journal of Graph Algorithms and Applications*, 10(1):5–49, 2006. doi: 10.7155/jgaa.00117.
- [3] L. Sunil Chandran and T. Kavitha. The treewidth and pathwidth of hypercubes. *Discrete Mathematics*, 306(3):359–365, 2006. doi: 10.1016/j.disc.2005.12.011.
- [4] Geoffrey Chu and Peter J. Stuckey. Minimizing the maximum number of open stacks by customer search. In *Principles and Practice of Constraint Programming – CP 2009*, volume 5732 of *Lecture Notes in Computer Science*, pages 242–257. Springer, 2009. doi: 10.1007/978-3-642-04244-7_21.
- [5] Enrico Faggioli and Carlo Alberto Bentivoglio. Heuristic and exact methods for the cutting sequencing problem. *European Journal of Operational Research*, 110(3):564–575, 1998. doi: 10.1016/S0377-2217(97)00269-7.
- [6] Michael R. Fellows and Michael A. Langston. Nonconstructive advances in polynomial-time complexity. *Information Processing Letters*, 26(3):157–162, 1987. doi: 10.1016/0020-0190(87)90008-5.
- [7] Michael R. Fellows and Michael A. Langston. On search, decision, and the efficiency of polynomial-time algorithms. In *Proceedings of the 21st Annual ACM Symposium on Theory of Computing (STOC)*, pages 501–512, 1989. doi: 10.1145/73007.73055.
- [8] Rafael de Magalhães Dias Frinhani, Marco Antonio Moreira de Carvalho, and Nei Yoshihiro Soma. A PageRank-based heuristic for the minimization of open stacks problem. *PLoS ONE*, 13(8):e0203076, 2018. doi: 10.1371/journal.pone.0203076.
- [9] L. H. Harper. Optimal numberings and isoperimetric problems on graphs. *Journal of Combinatorial Theory*, 1(3):385–393, 1966. doi: 10.1016/S0021-9800(66)80059-5.
- [10] Nancy G. Kinnersley. The vertex separation number of a graph equals its path-width. *Information Processing Letters*, 42(6):345–350, 1992. doi: 10.1016/0020-0190(92)90234-M.
- [11] Lefteris M. Kirousis and Christos H. Papadimitriou. Searching and pebbling. *Theoretical Computer Science*, 47:205–218, 1986. doi: 10.1016/0304-3975(86)90146-5.
- [12] Thomas Lengauer. Black-white pebbles and graph separation. *Acta Informatica*, 16(4):465–475, 1981. doi: 10.1007/BF00264496.
- [13] Alexandre Linhares and Horacio Hideki Yanasse. Connections between cutting-pattern sequencing, VLSI design, and flexible machines. *Computers & Operations Research*, 29(14):1759–1772, 2002. doi: 10.1016/S0305-0548(01)00054-5.
- [14] Rolf H. Möhring. Graph problems related to gate matrix layout and PLA folding. In *Computational Graph Theory*, volume 7 of *Computing Supplementum*, pages 17–51. Springer-Verlag, Wien, 1990. doi: 10.1007/978-3-7091-9076-0_2.

- [15] Tatsuo Ohtsuki, Hajimu Mori, Ernest S. Kuh, Toshinobu Kashiwabara, and Toshio Fujisawa. One-dimensional logic gate assignment and interval graphs. *IEEE Transactions on Circuits and Systems*, 26(9):675–684, 1979. doi: 10.1109/TCS.1979.1084695.
- [16] Nathan Wetzler, Marijn J. H. Heule, and Warren A. Hunt. DRAT-trim: Efficient checking and trimming using expressive clausal proofs. In *Theory and Applications of Satisfiability Testing – SAT 2014*, volume 8561 of *Lecture Notes in Computer Science*, pages 422–429. Springer, 2014. doi: 10.1007/978-3-319-09284-3_31.
- [17] Omar Wing, S. Huang, and R. Wang. Gate matrix layout. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 4(3):220–231, 1985. doi: 10.1109/TCAD.1985.1270118.
- [18] Horacio Hideki Yanasse. A transformation for solving a pattern sequencing problem in the wood cut industry. *Pesquisa Operacional*, 17(1):57–70, 1997.
- [19] Horacio Hideki Yanasse and Edson Luiz França Senne. The minimization of open stacks problem: A review of some properties and their use in pre-processing operations. *European Journal of Operational Research*, 203(3):559–567, 2010. doi: 10.1016/j.ejor.2009.09.017.
- [20] Horacio Hideki Yanasse, José Carlos Becceneri, and Nei Yoshihiro Soma. Bounds for a problem of sequencing patterns. *Pesquisa Operacional*, 19(2):249–277, 1999.

A Where every number comes from

Every number in the paper is copied from a numbered section of `reports/ml_nature.md`, summarised one paragraph per section in `reports/ml_nature_summary.md`, and regenerates by the command listed there. The table maps each claim to its section and command; the section holds the full form with flags and timings. Nothing in this paper was computed for the paper.

Table 8: Provenance of the paper’s numbers.

claim	section	regenerate
3,667 distinct graphs; 42.5% copies; 1,669 complete; 157 classes across files; 6,376 at 9–134	[§ 1]	<code>python -m learning.canonical -workers 16</code>
Fingerprint 94.4% / 98.8% vs 61.6%; 74 cells; SCOOP with Chu & Stuckey; Figure 1	[§ 2]	<code>python -m learning.fingerprint</code>
Treewidth heuristic 85.7% / 0.188; bounds 84.6% / 0.239, 77.0% / 0.431; below 481, above 428; MAE −0.004	[§ 4]	<code>python -m learning.dataset -workers 16 && python -m learning.invariants_study -workers 16</code>
Sandwich on 6,376; 2,823 forced; λ by density	[§ 5, 12]	<code>python -m learning.formula_search -workers 16; python -m learning.concentration</code>
Bound tight 77.0%, by one 17.7%, 338 gap (5.3%, 51.0% of C&S); no gap below 20; 131 certified; family	[§ 6]	<code>python -m learning.bound_gap -workers 16</code>
Rule 0.348 / 80.7% / +9; ranker 0.519 / 75.0% / +34; MCN 1.616 / 50.2% / +26; 1,920 held out	[§ 7]	<code>python -m learning.distil -workers 16</code>
0 of 2,812 unique; 4×10^5 at 10, 2.5×10^{10} at 15; 90% of 2.27 M decisions	[§ 8]	<code>python -m learning.degeneracy -workers 16</code>
Campaign 37,800 in 252 cells, 150 per cell, 2.16 core-hours; refutation 0.5–0.7% of cost; three orders of magnitude at $n = 40$	[§ 9, 10]	<code>python -m learning.ensemble -item02 -workers 16</code>
Ridge: two orders each side; 0.086–0.105 per customer; doubling 2.9–3.5 / 4.5–4.9; three and two customers per product; C&S density 2 on the peak; 5×10^{11} ; Figure 3	[§ 11]	<code>python -m learning.hardness_map</code>

claim	section	regenerate
CV < 0.2 in 36 cells, median 0.031; $r^2 \geq 0.999$ in 17 series; MAE 0.49; below 9,445, above 10,641; Figure 2	[§ 12]	python -m learning.concentration -workers 4
15,900 re-coverings unchanged; relabelling 2–6%, 2.26×, 8.1×	[§ 13]	python -m learning.graph_story -workers 16 -per-n 200 -per-method 4 -relabellings 20
Treewidth heuristic 4.2 off at 125, 2 of 25; bias +8.1, ± 2 ; intervals fail; $1.6\text{--}1.7 \times 10^{11}$ record; Figure 4	[§ 14]	python -m learning.scale_test
43,935 instances; 878,580 refutations; 88 runs on 56 instances; fix +7.3%	[§ 15]	python -m learning.differential -workers 16
6,747 certified at 50–75; drift 0.002 per ten customers; 18 of 18; 1.5×10^{11}	[§ 16]	python -m learning.ensemble -upward; python -m learning.upward
5,646 of 6,135 (92.0%); 99.4 / 97.1 / 80.0 / 77.7%; 11.2 core-hours; 11.7 GB; $m^2k \approx 10^4$; 6,000–70,000×	[§ 17]	python -m learning.proofs run -max-customers 40 -workers 16 -conflicts 200000
Min-of-16 1.00–1.02; spread 1.12 / 1.02 / 1.02 / 1.01; seconds 1.49× vs nodes 1.05×	[§ 18]	python -m learning.relabel_portfolio -workers 16 -deadline 120
87.8% of 98, 89.3% of 103 within a decade; baselines 57.5–65.4%, 59.2–61.2%; bias +0.4	[§ 19]	python -m learning.cost_model
Fan order 0.0%; 35% of witness nodes at 75	[§ 20]	python -m learning.fan_order -workers 16
10-vertex graph, 21 edges, tw 3, pw 5; 164 / 80 / 94 of 338; exact treewidth to 26 and 64	[§ 21]	python -m learning.extremal -stage corpus -workers 8
Theorem 2: –6.6% nodes; one cost; 1.167× per node; slower in seconds	[§ 22]	python -m learning.theorem2 -workers 16
Set-valued policy 0.008 vs 0.02; 99.71%	[§ 23]	python -m learning.set_imitation -stage tables
933 (14.6%), 199 of 338; 11,589 candidates, 47 survivors, 36 restated; 77% leak; 0.1%	[§ 24]	python -m learning.conjecture -stage tables
Mean degree 3.9–37.6; excess 2.0–2.5 / 1.8–3.0; factor 1.65; $1 + 2.4n/m$; Figure 5	[§ 25]	python -m learning.ridge_theory -stage tables
Both sandwich ends theorems; star counterexample; equality proved; 34 theorems, one sorry	[§ 26, 39, 40]	cd lean && lake build; python -m learning.sandwich -stage lean
287,884 graphs on ≤ 9 ; four on 10; 1,034 on 11, none gap 3; twelve with tw 2, pw 4	[§ 27]	python -m learning.pwtw_exhaust -stage tables
rule+cs-dfs 94.2% / 0.092 / 16.7 ms; learned 92.1% / 0.128 / 20.6; cs-dfs 84.5% / 0.241 / 29.5; a seventh of the gain was the split; 5,938 at ≤ 30 ; 25 rows	[§ 28]	python -m learning.rule_seed -workers 16
MCNh 21 of 21; 98.8%; 0.6%; 592 / 242	[§ 29]	python -m learning.mcnh -workers 16; -stage paper
SAT conflicts 0.26–1.67×; Spearman 0.93–0.98; 5,657 decisions agree	[§ 30]	python -m learning.sat_story -stage tables
Reverts 1 / 36 of 56; 1.073×, 81%; 1.54× at 41–100; ≥ 19.6 ×; bm-first 0.854×, 0.98	[§ 31]	python -m learning.fix_cost -stage tables
6,135 certificates, 0 rejected; 5.0 MB, 4.5 s vs 11.7 GB, 5,190 s; 141 of 151; 120×; 10^{11} nodes, 10TB	[§ 32]	python -m learning.search_certificate -stage tables
1,294 instances; 25,800 calls; 12,860 / 12,860; 12,938 / 12,940; eight priced out	[§ 33]	python -m learning.differential_scale -stage tables
100 cell: 24 of 25 censored at 2,400 s, $3.4\text{--}4.8 \times 10^9$; one certified; 0.5–0.7 decades; 1.1 decades	[§ 34]	python -m learning.ridge100 -stage tables
Rate 0.116 [0.110, 0.121] vs 0.092–0.095; 2.7×10^{11} [2.3, 3.2]; 55 h; ≥ 2.34 ×; two withdrawn instances	[§ 35]	python -m learning.rate_drift -stage tables
Giant component 0.34–0.78 decades below; reachable sets 0.03–0.06 decades	[§ 36]	python -m learning.cover_excess -stage tables
Height law; RMSE 0.18 / 0.21; 57 peaks; 3,378 certified	[§ 37]	python -m learning.ridge_height -stage tables

B Solver changes proposed, and the two applied

A reader may want to know what in this paper is a finding and what is a recommendation. Every entry below was measured behind a flag that defaults to today’s behaviour, or in a report, and none was enabled by a loop; enabling any of them is the owner’s change. Two were applied by the owner: the `better_move` fix, and the two-key rule as the default start (entries 3 and 14). The full table with the measurement that decides each entry is section 8 of the summary.

Table 9: Proposed solver changes, from the summary’s section 8.

#	change	measured effect	state
1	Fix the C <code>better_move</code> [§ 15, 31]	false refutations at the optimum on 56 of 43,935 instances; fix costs +7.3% nodes at ≤ 40 , $1.54\times$ at 41–100	applied by the owner
2	Exact or interval treewidth as a lower-bound component [§ 24]	floor rises on 933 instances; valid on all 50,949	proposed
3	Two-key rule as heuristic and DFS seed [§ 7, 28]	780 better / 12 worse over cs-dfs on 6,376	applied by the owner, 2026-09-28
4	Degree fan order on <code>decide</code> [§ 20]	0.0% on refutations	closed
5	Degree fan order on satisfiable-side drivers [§ 20]	–35% witness nodes at 75, –13% at 100	proposed
6	cs-dfs+degree as the cs-dfs default [§ 20, 28]	128 / 113 vs cs-dfs at $1.5\times$ the time	registered, not default
7	Theorem 2 always on [§ 22]	–6.6% nodes, +5% seconds	recommended against
8	Relabelling portfolio on refutations [§ 18, 33]	$1.00\text{--}1.01\times$	recommended against
9	Relabelling portfolio on satisfiable calls [§ 18, 33]	$8\text{--}13\times$ where the witness search is hard; 2–5% of a descent	proposed
10	Cost model to order the recertify queue [§ 19, 34]	88–89% within a decade	proposed, never for k
11	learned+cs-dfs as default upper bound	dominated by the rule seed 263 / 80	registered, not default
12	Group study splits by graph certificate [§ 1]	157 classes leak across files	proposed (study change)
13	Draw the dominance test’s instances sparse; add the two minimal counterexamples [§ 15]	the dense family never exposed the cycle	proposed
14	rule+cs-dfs wherever cs-dfs is used [§ 28]	627 of 988 misses recovered; 16.7 vs 29.5 ms	applied by the owner, 2026-09-28
15	<code>mcnh</code> strategies [§ 29]	the published MCNh to within tie-breaks	registered as reference point
16	bm-first composition of the dominance rules [§ 31]	sound where measured; $0.854\times$ nodes at 41–100; seconds 0.98	proposed to tune, not adopt
17	Pre-encoding re-cover for the SAT path [§ 30]	$0.77\times$ conflicts, $0.71\times$ seconds at the median	proposed, to be raced
18	C emitter for the search certificate [§ 32]	Python emitter $120\times$ slower than the C	proposed
19	recertify emits and checks a certificate [§ 32]	5.0 MB and 4.5 s for the corpus at ≤ 40	proposed
20	Lean statement of the three dominance theorems [§ 32]	the one trusted step the checker leaves	proposed
21	Price-stage admission rule for harness runs [§ 33]	price / actual 1.38 over 25,800 calls	proposed
22	Certificates for the eight priced-out instances [§ 33]	needs #18	proposed